Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Kaski

Lab 2: Implementation of Source Code Management Using Git Commands Through GitHub — Build, Breakpoints, Refactoring, Collaboration, Copilot, IntelliSense

Submitted By:

Abhipsa Buddhacharya

BESE2023

Roll no. 5

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE:

Implementation of Source Code Management Using Git Commands Through GitHub — Build, Breakpoints, Refactoring, Collaboration, Copilot, IntelliSense

AIM:

2a: Build – Release and Debug

To understand and implement build configurations in software development, specifically differentiating between Debug and Release build modes, and to observe their effects on application behavior and performance.

THEORY:

A build configuration defines how source code is compiled and linked into an executable. The two primary configurations used in software development are Debug and Release.

Debug Build: Includes full symbolic debugging information, disables compiler optimizations, and enables runtime checks. This makes the executable larger and slower but easier to trace and inspect during development.

Release Build: Applies compiler optimizations, strips debugging symbols, and produces a smaller, faster executable suitable for deployment to end users.

In VS Code, build configurations are managed through launch.json and tasks.json files inside the .vscode folder.

OBJECTIVES:

- Understand the difference between Debug and Release build configurations.
- Configure a project for Debug and Release builds in VS Code.
- Compile and run the project in both configurations.
- Compare output, file size, and performance between the two builds.

ALGORITHM:

1. Create a new project folder and open it in VS Code.
2. Write a sample program (e.g., a simple calculator in C++ or Node.js).
3. Create .vscode/tasks.json and configure a Debug build task.
4. Create a Release build task with optimization flags.

5. Run the Debug build and observe the output.
6. Run the Release build and compare file sizes and execution time.
7. Document observations.

CODE:

Sample C++ program (main.cpp):

```
#include <iostream>

using namespace std;



int main() {

    int a = 10, b = 5;

    cout << "Sum: " << a + b << endl;

    cout << "Product: " << a * b << endl;

    return 0;

}
```

Build tasks (tasks.json):

```
{

  "version": "2.0.0",

  "tasks": [

    { "label": "Build Debug",   "type": "shell", "command": "g++ -g  -o debug_out
main.cpp" },

    { "label": "Build Release", "type": "shell", "command": "g++ -O2 -o release_out
main.cpp" }

  ]

}
```

OUTPUT:

```
Sum: 15
Product: 50
```

- Debug binary: larger size, contains symbol table.
- Release binary: smaller size, faster execution, no debug symbols.

RESULT:

Both Debug and Release builds were successfully compiled and executed. The Debug build retained full symbol information useful for troubleshooting, while the Release build produced an optimized binary for production.

CONCLUSION:

This experiment demonstrated the importance of build configurations in the software development lifecycle. Debug builds support development and testing, while Release builds are optimized for end-user deployment.

2b: Breakpoints

AIM :

To understand and apply breakpoints in VS Code for debugging software, including setting, managing, and using conditional breakpoints to inspect program state during execution.

THEORY :

A breakpoint is a debugging tool that pauses program execution at a specific line, allowing the developer to inspect variables, call stacks, and program flow at that moment.

Types of breakpoints in VS Code:

- Line Breakpoint: Pauses execution at a specific line number.
- Conditional Breakpoint: Pauses only when a specified condition is true (e.g., i == 5).
- Logpoint: Logs a message to the console without stopping execution.
- Exception Breakpoint: Pauses when an exception is thrown.

OBJECTIVES:

- Set a basic line breakpoint and observe program pause.
- Inspect variable values at the breakpoint.
- Use conditional breakpoints.
- Use the Watch panel and Call Stack during debugging.

ALGORITHM :

1. Open a JavaScript or Python project in VS Code.
2. Write a loop-based program with a variable to watch.
3. Click the gutter (left of line number) to set a breakpoint.
4. Open the Run and Debug panel (Ctrl+Shift+D).
5. Start debugging (F5).
6. Inspect variables in the Variables panel.
7. Set a conditional breakpoint (right-click gutter > Edit Breakpoint).
8. Continue and verify the condition triggers correctly.

CODE:

Sample JavaScript program (debug_demo.js):

```
function calculateSum(n) {
    let sum = 0;
    for (let i = 1; i <= n; i++) {
        sum += i;  // Set breakpoint here
    }
    return sum;
}
console.log('Sum:', calculateSum(10));
```

launch.json configuration:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Debug Demo",
      "program": "${workspaceFolder}/debug_demo.js"
    }
  ]
}
```

OUTPUT :

- Execution paused at the breakpoint on the sum += i line.
- Variables panel showed current values of i and sum at each iteration.
- Conditional breakpoint (i === 5) triggered only when i equaled 5.

```
Sum: 55
```

RESULT:

Breakpoints were successfully set and used to pause, inspect, and trace program execution in VS Code.

CONCLUSION :

Using VS Code's debugger with line and conditional breakpoints enables developers to efficiently trace logic errors and inspect runtime state — a critical practice in Agile iterative development.

2c: Refactoring

AIM :

To understand and apply code refactoring techniques using VS Code's built-in refactoring tools to improve code readability, maintainability, and structure without changing its external behavior.

THEORY:

Refactoring is the process of restructuring existing code without altering its external behavior. Common techniques include:

- Rename Symbol: Renaming a variable or function across the codebase (F2 in VS Code).
- Extract Function: Moving a block of code into a separate named function.
- Extract Variable: Replacing a repeated expression with a named variable.
- Inline Variable: Replacing a variable with its direct value where used once.

OBJECTIVES:

- Identify code that needs refactoring.
- Apply rename, extract function, and extract variable refactoring in VS Code.
- Verify that program behavior remains unchanged after refactoring.

ALGORITHM:

9. Open a JavaScript project in VS Code with poorly structured code.
10. Identify a long function that does multiple things.
11. Select a block of code and use Refactor > Extract Function.
12. Rename a poorly named variable using F2 > Rename Symbol.
13. Run the program before and after to confirm identical output.
14. Document the before and after code structure.

CODE:

Before refactoring:

```
function process(a, b) {
    let x = a * a + b * b;
    let y = Math.sqrt(x);
    console.log('Result:', y);
    let z = a + b;
    console.log('Sum:', z);
}
```

After refactoring:

```
function calculateHypotenuse(sideA, sideB) {
    return Math.sqrt(sideA ** 2 + sideB ** 2);
}
function calculateSum(sideA, sideB) {
    return sideA + sideB;
}
function process(sideA, sideB) {
    console.log('Hypotenuse:', calculateHypotenuse(sideA, sideB));
    console.log('Sum:', calculateSum(sideA, sideB));
}
```

OUTPUT:

```
Hypotenuse: 5
Sum: 7
```

- Code readability improved significantly.
- Each function now has a single, clear responsibility.

RESULT:

Code refactoring was successfully performed using VS Code's built-in tools. The refactored code produced identical output while being significantly more readable and maintainable.

CONCLUSION:

VS Code's refactoring tools make it efficient to rename, extract, and restructure code safely, reducing technical debt and supporting long-term project health in Agile development.

2d: Source Code Collaboration

AIM:

To implement collaborative software development practices using GitHub, including feature branching, pull requests, and code review workflows as practiced in Agile teams.

THEORY:

Source code collaboration allows multiple developers to work on the same codebase concurrently. GitHub facilitates this through:

- Branching: Creating isolated environments for new features without affecting the main codebase.
- Pull Requests (PRs): Formal requests to merge a feature branch into main, enabling code review.
- Code Review: Team members review changes, leave comments, and approve PRs before merging.
- Fork and PR Workflow: External contributors fork a repo and submit PRs — the standard open-source model.

OBJECTIVES:

- Create and switch between Git branches.
- Make changes on a feature branch and push to GitHub.
- Open a Pull Request and simulate a code review.
- Merge the feature branch into main after review.

ALGORITHM:

15. Clone the shared repository.
16. Create a new feature branch: git checkout -b feature/my-feature.
17. Make changes and commit.
18. Push the branch to GitHub.
19. Open a Pull Request on GitHub targeting the main branch.
20. Request a review and address comments.
21. Merge the PR after approval.

CODE:

Create and switch to a feature branch:

```
git checkout -b feature/add-login
```

Stage, commit, and push:

```
git add .
git commit -m "Add login functionality"
git push origin feature/add-login
```

Merge after approval:

```
git checkout main
git merge feature/add-login
git push origin main
```

OUTPUT :

- Feature branch created and pushed successfully.
- Pull Request opened on GitHub with changes visible in the diff view.
- Code review comments added and resolved.

- PR merged into main branch — no conflicts.

RESULT:

Source code collaboration using GitHub's branching and Pull Request workflow was successfully implemented.

CONCLUSION:

The PR-based workflow is the industry standard for both open-source and professional software development, enabling Agile teams to work in parallel and maintain a clean main branch.

2e: GitHub Copilot, Code Generation, and IntelliSense

AIM:

To explore and implement AI-assisted code generation using GitHub Copilot and IntelliSense in VS Code, and to understand how these tools enhance developer productivity in an Agile environment.

THEORY:

IntelliSense is VS Code's built-in code completion system. It provides real-time suggestions including autocompletion, parameter hints, quick documentation, and go-to-definition.

GitHub Copilot is an AI-powered code generation tool that uses a large language model to suggest entire lines, functions, or blocks of code based on context and natural language comments.

- IntelliSense: Autocompletes variable names, function signatures, and imports as you type.
- GitHub Copilot: Generates function bodies from comments or partial code.
- Context-Aware Suggestions: Copilot adapts suggestions based on existing code style and imports.

OBJECTIVES:

- Configure IntelliSense for a JavaScript project in VS Code.
- Install and activate GitHub Copilot in VS Code.
- Use Copilot to generate functions from natural language comments.

- Compare IntelliSense completions vs Copilot suggestions.

ALGORITHM:

22. Open VS Code and create a new JavaScript file.
23. Observe IntelliSense triggering automatically as you type (Ctrl+Space to force).
24. Install GitHub Copilot extension from the VS Code Marketplace.
25. Sign in with a GitHub account that has Copilot access.
26. Write a comment describing a function you want.
27. Press Enter and observe Copilot's suggestion (grey ghost text).
28. Accept the suggestion with Tab.
29. Run the generated function and verify correctness.

CODE:

IntelliSense – typing Math. triggers suggestions:

```
const x = Math.   // IntelliSense shows: abs, ceil, floor, round, sqrt...
```

GitHub Copilot – comment-driven code generation:

```
// Function to check if a number is prime
function isPrime(n) {
    if (n < 2) return false;
    for (let i = 2; i <= Math.sqrt(n); i++) {
        if (n % i === 0) return false;
    }
    return true;
}  // Copilot generated this from the comment above
```

Testing the generated function:

```
console.log(isPrime(7));   // true
console.log(isPrime(10));  // false
```

OUTPUT:

- IntelliSense provided real-time completions for Math methods and function parameters.
- Copilot generated a complete and correct isPrime function from a single comment.

```
true
false
```

RESULT:

AI-assisted code generation using GitHub Copilot and IntelliSense was successfully demonstrated. Copilot generated correct implementations from natural language comments while IntelliSense provided continuous context-aware completions.

CONCLUSION:

GitHub Copilot and IntelliSense significantly accelerate development by reducing repetitive typing and suggesting correct implementations. Generated code should always be reviewed and tested before committing.